

Pandas Library Python Notes

Introduction:

Pandas is a powerful, open-source library for data analysis and manipulation in Python.

- It provides powerful tools to work with structured data, such as tables (like an Excel sheet or a database table).
- It has functions for analyzing, cleaning, exploring, and manipulating data.
- The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008

What Can Pandas Do?

Pandas gives you answers about the data. Like:

- Is there a correlation between two or more columns?
- What is average value?
- Max value?
- Min value?

Pandas are also able to delete rows that are not relevant, or contains wrong values, like empty or NULL values. This is called *cleaning* the data.

Why Use Pandas?

- Handles large amounts of data efficiently
- Easy to clean, transform, and analyze data
- Works well with other libraries like NumPy and Matplotlib

Installing Pandas

Before using Pandas, you need to install it. Run this command in your terminal or command prompt:

- `pip install pandas`

Once installed, import it in Python:

- `import pandas as pd`

Checking the Pandas Version

If you want to check which version of Pandas is installed on your system, use:

1. `import pandas as pd`
2. `print(pd.__version__)`

Pandas Data Structures

Pandas has two main data structures:

1. **Series** – A one-dimensional (1D) array, like a list or column of data.
2. **DataFrame** – A two-dimensional (2D) table, like an Excel sheet with rows and columns.

Series

A **Series** is like a column in a table. You can create a Series using a list:

1. `import pandas as pd`
2. `data = [10, 20, 30, 40]`
3. `series = pd.Series(data)`
4. `print(series)`

Note: The numbers **0, 1, 2, 3** in the output are the **index** (row labels), and the values are stored in the Series.

DataFrame

A **DataFrame** is like an Excel table with rows and columns.

1. `data = {
 "Name": ["Alice", "Bob", "Charlie"],
 "Age": [25, 30, 35],
 "City": ["New York", "London", "Paris"]
}`
2. `df = pd.DataFrame(data)`
3. `print(df)`

Reading & Writing Data

What is a CSV File?

A **CSV (Comma-Separated Values) file** is a simple text file used to store tabular data (like a spreadsheet or database) in a structured format. Each line in the file represents a row, and the values in each row are separated by commas.

Example of a CSV File

Here's what a CSV file (data.csv) looks like:

- Name, Age, City
- Alice, 25, New York
- Bob, 30, London
- Charlie, 35, Paris

This file has:

- **Columns:** Name, Age, City
- **Rows:** Three people's information

Why Use CSV Files?

- Easy to create and read
- Works with Excel, databases, and programming languages
- Small file size compared to other formats like Excel (.xlsx)

How to Open a CSV File?

You can open a CSV file with:

- **Excel** (automatically formats the data into columns)
- **Notepad** or any text editor
- **Python (Pandas or CSV module)**

Reading CSV Files

To read a CSV file:

```
df = pd.read_csv("data.csv")  
print(df)
```

Reading a CSV File in Python

Using **Pandas**:

1. import pandas as pd
2. df = pd.read_csv("data.csv")
3. print(df)

Using **CSV Module**:

1. import csv

```
2. with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

Writing to a CSV File

To save a DataFrame to a CSV file:

1. `df.to_csv("output.csv", index=False)`
2. `index=False` prevents Pandas from saving the index column.

Basic Operations on DataFrames

Viewing Data

- **First few rows:**

1. `print(df.head())` # Shows first 5 rows
2. `print(df.head(10))` # Shows first 10 rows

- **Last few rows:**

1. `print(df.tail())` # Shows last 5 rows

4.2 Getting Data Info

- **Summary of the dataset:**

1. `print(df.info())`

- **Statistics of numeric columns:**

1. `print(df.describe())`

4.3 Selecting Columns

To get a single column:

```
1. print(df["Name"])
```

To get multiple columns:

```
1. print(df[["Name", "Age"]])
```

4.4 Selecting Rows

- **By index:**

```
1.print(df.loc[0]) # First row
2.print(df.iloc[2]) # Third row
```

- **By condition:**

```
1.print(df[df["Age"] > 25]) # People older than 25
```

Adding a Column

```
1. df["Salary"] = [50000, 60000, 70000]
2. print(df)
```

Removing a Column

```
1. df = df.drop(columns=["Salary"])
```

Removing Rows

```
1. df = df.drop(index=1) # Removes row at index 1
```

Handling Missing Data

Checking for Missing Values

```
1. print(df.isnull().sum()) # Shows count of missing values
```

Filling Missing Values

```
1. df.fillna("Unknown", inplace=True) # Replaces NaN with "Unknown"
```

Dropping Rows with Missing Values

```
1. df.dropna(inplace=True)
```

Sorting & Filtering Data

Sorting Data

```
1. df = df.sort_values("Age") # Sorts by Age (ascending)
2. df = df.sort_values("Age", ascending=False) # Sorts by Age (descending)
```

Filtering Data

```
1. df_filtered = df[df["City"] == "London"] # Filters only London
```

Grouping & Aggregation

Pandas allows us to group data and apply functions like `sum()`, `mean()`, etc.

```
1. df_grouped = df.groupby("City")["Age"].mean()
2. print(df_grouped)
```

Merging & Joining DataFrames

Merging DataFrames

1. `df1 = pd.DataFrame({"ID": [1, 2, 3], "Name": ["A", "B", "C"]})`
2. `df2 = pd.DataFrame({"ID": [1, 2, 3], "Salary": [1000, 2000, 3000]})`
3. `merged_df = pd.merge(df1, df2, on="ID")`
4. `print(merged_df)`

Joining DataFrames

1. `df1.set_index("ID", inplace=True)`
2. `df2.set_index("ID", inplace=True)`
3. `joined_df = df1.join(df2)`
4. `print(joined_df)`

Applying Functions to Data

Using `apply()` Function

1. `df["Age_Group"] = df["Age"].apply(lambda x: "Young" if x < 30 else "Old")`

Using `map()` Function

1. `df["Age"] = df["Age"].map(lambda x: x + 5) # Increases age by 5`

Visualization with Pandas

Pandas works with Matplotlib for quick visualizations.

1. `import matplotlib.pyplot as plt`
2. `df["Age"].plot(kind="bar")`
3. `plt.show()`

`pivot_table()` - Summarizing Data

The `pivot_table()` function helps you **summarize** and **aggregate** data in a structured way. It's useful when dealing with large datasets.

Example: Creating a Pivot Table

1. `import pandas as pd`
2. `# Creating a dataset`
3. `data = {`
 `'Employee': ['Alice', 'Bob', 'Alice', 'Bob', 'Charlie', 'Charlie'],`
 `'Department': ['HR', 'IT', 'HR', 'IT', 'Finance', 'Finance'],`
 `'Salary': [50000, 60000, 52000, 61000, 70000, 71000]`
 `}`
4. `df = pd.DataFrame(data)`
5. `# Creating a pivot table to calculate the average salary per department`
6. `pivot = df.pivot_table(values='Salary', index='Department', aggfunc='mean')`
7. `print(pivot)`

Output:

<i>Department</i>	<i>Salary</i>
Finance	70500.0
HR	51000.0
IT	60500.0

- `values='Salary'` → The column we want to summarize.
- `index='Department'` → Groups the data by the Department.
- `aggfunc='mean'` → Calculates the average salary for each department.

`crosstab()` - Frequency Tables

The `crosstab()` function is used to count how many times values appear in different categories. It's useful for analyzing relationships between two columns.

Example: Creating a Crosstab Table

1. `import pandas as pd`
2. `# Creating a dataset`
3. `data = {`
 `'Employee': ['Alice', 'Bob', 'Alice', 'Bob', 'Charlie', 'Charlie'],`
 `'Department': ['HR', 'IT', 'HR', 'IT', 'Finance', 'Finance']`
 `}`
4. `df = pd.DataFrame(data)`
5. `# Creating a crosstab to count employees in each department`
6. `cross_tab = pd.crosstab(df['Employee'], df['Department'])`
7. `print(cross_tab)`

Output:

Department	Finance	HR	IT
Employee			
Alice	0	2	0
Bob	0	0	2
Charlie	2	0	0

- This table shows how many times each employee appears in each department.

`melt()` - Reshaping Data

The `melt()` function is used to **transform wide data into long format**, making it easier to analyze.

Example: Melting a DataFrame

1. `import pandas as pd`
2. `# Creating a dataset`
3. `data = {`
 `'Employee': ['Alice', 'Bob', 'Charlie'],`
 `'Salary_2023': [50000, 60000, 70000],`
 `'Salary_2024': [52000, 61000, 71000]`
 `}`
4. `df = pd.DataFrame(data)`

5. # Melting the DataFrame to reshape it
6. melted = pd.melt(df, id_vars=['Employee'], var_name='Year', value_name='Salary')
7. print(melted)

Output:

```
Employee  Year  Salary
0  Alice  Salary_2023  50000
1    Bob  Salary_2023  60000
2  Charlie  Salary_2023  70000
3    Alice  Salary_2024  52000
4     Bob  Salary_2024  61000
5  Charlie  Salary_2024  71000
```

- **Before melting**, the salaries for different years were in separate columns.
After melting, the data is transformed into a format where each row represents a salary for a specific year.
-

Work with Real-World Datasets

Instead of working with small sample datasets, you can start using **real-world data** stored in different formats like:

1. **CSV Files**
2. **Excel Sheets**
3. **Databases (SQL, MongoDB, etc.)**

Reading a CSV File

1. import pandas as pd
 2. # Reading a CSV file
 3. df = pd.read_csv('sales_data.csv')
 4. # Display the first 5 rows
 5. print(df.head())
- .head() shows the first 5 rows of the dataset.

Reading an Excel File

1. df = pd.read_excel('sales_data.xlsx')
- Requires pip install openpyxl to read Excel files.

Reading from a SQL Database

1. import sqlite3
2. # Connecting to a database
3. conn = sqlite3.connect('sales.db')
4. # Reading data from a table into a Pandas DataFrame
5. df = pd.read_sql_query("SELECT * FROM sales", conn)
6. print(df.head())

- This allows Pandas to work with SQL databases directly.

Data Visualization with Pandas, Matplotlib, and Seaborn 🧠

Pandas integrates with **Matplotlib** and **Seaborn** to visualize data.

Line Chart with Matplotlib

```
1. import pandas as pd
2. import matplotlib.pyplot as plt
3. # Creating a dataset
   data = {
       'Year': [2020, 2021, 2022, 2023, 2024],
       'Sales': [100, 150, 200, 250, 300]
   }
4. df = pd.DataFrame(data)
5. # Plotting a line graph
6. plt.plot(df['Year'], df['Sales'], marker='o', linestyle='-')
7. plt.xlabel('Year')
8. plt.ylabel('Sales')
9. plt.title('Sales Over Time')
10. plt.show()
```

- This creates a **line chart** showing sales growth over time.

Bar Chart with Seaborn

```
1. import seaborn as sns
2. # Creating a bar chart
3. sns.barplot(x=df['Year'], y=df['Sales'])
4. plt.title('Sales Growth')
5. plt.show()
```

What is JSON?

JSON (JavaScript Object Notation) is a **text-based format** used to store and transfer data. It is widely used in programming and is **similar to Python dictionaries**.

Why Use JSON in Pandas?

- ✓ JSON is **lightweight** and easy to read.
- ✓ It is commonly used in **APIs** and **big data** applications.
- ✓ Pandas can **directly read** JSON files and convert them into a DataFrame for easy analysis.

Reading a JSON File in Pandas

In this example, we have a **JSON file** called "**data.json**". Let's load it into a Pandas DataFrame.

Example: Reading a JSON File

```
1. import pandas as pd
2. # Read JSON file into a DataFrame
3. df = pd.read_json('data.json')
4. # Print the full DataFrame
5. print(df.to_string())
```

Explanation:

- `pd.read_json('data.json')` → Reads the JSON file and converts it into a **DataFrame**.
- `print(df.to_string())` → Displays the entire table neatly.

Sample JSON File (data.json)

```
{
  "Name": ["Alice", "Bob", "Charlie"],
  "Age": [25, 30, 35],
  "City": ["New York", "London", "Paris"]
}
```

Output in DataFrame Format

	Name	Age	City
0	Alice	25	New York
1	Bob	30	London
2	Charlie	35	Paris

JSON as a Python Dictionary

JSON objects have the **same format** as Python **dictionaries**. If the JSON data is not in a file but is stored as a dictionary, we can **directly convert it into a DataFrame**.

◆ Example: Using a Python Dictionary as JSON

```
1. import pandas as pd
2. # Creating JSON data as a Python dictionary
3. data = {
    "Duration": { "0":60, "1":60, "2":60, "3":45, "4":45, "5":60 },
    "Pulse": { "0":110, "1":117, "2":103, "3":109, "4":117, "5":102 },
    "Maxpulse": { "0":130, "1":145, "2":135, "3":175, "4":148, "5":127 },
    "Calories": { "0":409, "1":479, "2":340, "3":282, "4":406, "5":300 }
  }
4. # Convert dictionary to DataFrame
5. df = pd.DataFrame(data)
6. # Print DataFrame
7. print(df)
```

Output

	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409
1	60	117	145	479

2	60	103	135	340
3	45	109	175	282
4	45	117	148	406
5	60	102	127	300

Explanation:

- The **Python dictionary** is structured like a JSON object.
- We use `pd.DataFrame(data)` to **convert** it into a Pandas **DataFrame**.
- Now we have **structured tabular data** for analysis!

Saving a DataFrame as JSON

Pandas also allows us to **save** a DataFrame as a JSON file using `to_json()`.

Example: Save DataFrame as JSON

1. `# Save DataFrame to JSON file`
2. `df.to_json('output.json')`

- This will create a **new JSON file** named `output.json` with the DataFrame data.

Viewing the Data in a DataFrame

`head()` - View the First Few Rows

The `.head()` method **shows the first few rows** of the DataFrame. By default, it shows **5 rows**, but you can specify any number.

Example: Show the First 10 Rows

1. `import pandas as pd`
2. `# Read data from a CSV file`
3. `df = pd.read_csv('data.csv')`
4. `# Display the first 10 rows`
5. `print(df.head(10))`

Explanation:

1. `pd.read_csv('data.csv')` → Loads data from a CSV file into a **DataFrame**.
2. `df.head(10)` → Displays the **first 10 rows** of the dataset.
3. If no number is given, `df.head()` **defaults to 5 rows**.

`tail()` - View the Last Few Rows

The `.tail()` method **shows the last few rows** of the DataFrame.

Example: Show the Last 5 Rows

1. # Display the last 5 rows
2. print(df.tail())

Explanation:

- df.tail(5) → Shows the **last 5 rows** (default).
- This helps check **recent** or **newly added data**.

Getting Information About the Data

info() - Get an Overview of the DataFrame

The .info() method provides **important details** about the dataset.

Example: Get Data Information

```
# Show DataFrame information
print(df.info())
```

Output Example:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 169 entries, 0 to 168
Data columns (total 4 columns):
#   Column   Non-Null Count  Dtype
---  -
0   Duration 169 non-null   int64
1   Pulse    169 non-null   int64
2   Maxpulse 169 non-null   int64
3   Calories 164 non-null   float64
```

What This Output Means:

- **169 rows** and **4 columns** in the dataset.
- **Column names:** Duration, Pulse, Maxpulse, Calories.
- **Data types:**
- int64 → Whole numbers (e.g., Duration, Pulse, Maxpulse).
- float64 → Decimal numbers (e.g., Calories).
- **Non-null values:**
- Calories has **164 values instead of 169**, meaning **5 rows have missing data**.

Handling Missing (Null) Values

- In the example above, the **Calories column** is missing some values.
- **Missing data can cause errors** in analysis, so we should **fix or remove** them.

How to Handle Missing Values?

- **Remove rows with missing values** (if they are not useful).
- **Fill missing values** with a default or average value.

Cleaning Data in Pandas

When working with real-world data, **errors and missing values** are common. Cleaning data is an **important step** before analysis.

Cleaning Empty Cells (Missing Data) ☹

Problem:

- Some data might be **missing** (empty cells), which can affect calculations and analysis.

Solution:

- We can either **remove** or **fill** empty cells.

Remove Rows with Missing Data

1. `import pandas as pd`
2. `df = pd.read_csv('data.csv')` # Load data
3. `df_cleaned = df.dropna()` # Remove rows with empty cells
4. `print(df_cleaned)`

- **dropna()** removes any row that has **empty values**.

Fill Empty Cells with a Default Value

1. `df_filled = df.fillna(0)` # Replace empty cells with 0
2. `print(df_filled)`

- **fillna(0)** fills empty cells with 0. You can also use **average values** instead.

1. `df['Calories'].fillna(df['Calories'].mean(), inplace=True)` # Fill with average

- This replaces missing **Calories** with the **average value** of the column.

Cleaning Wrong Format (Fix Data Types) 🛠

Problem:

- Sometimes, values are stored in the **wrong format** (e.g., dates stored as text).

Solution:

- Convert data to the correct format using **pd.to_datetime()** or **astype()**.

Convert Text to Date Format

1. `df['Date'] = pd.to_datetime(df['Date'])` # Convert text to date

- This ensures that dates are correctly formatted for analysis.

Convert Strings to Numbers

1. `df['Calories'] = pd.to_numeric(df['Calories'], errors='coerce')`

- Converts non-numeric values to numbers.
- **Invalid values** will be turned into NaN (missing values), which can be removed using `dropna()`.

Cleaning Wrong Data (Fixing Errors)

Problem:

Sometimes, **data contains mistakes**, like:

- Negative values where they don't make sense (e.g., negative age).
- Unrealistic values (e.g., 300 BPM heart rate).

Solution:

- Replace incorrect values with **valid ones**.

Fix Negative or Incorrect Values

1. `df.loc[df['Age'] < 0, 'Age'] = df['Age'].mean()` # Replace negative ages with the average age

- Finds rows where **Age < 0** and replaces them with the **average Age**.

Removing Duplicates (Same Data Multiple Times)

Problem:

- Sometimes, the same row appears **more than once** (duplicate data).

Solution:

- We can **remove duplicate rows** using `drop_duplicates()`.

Remove Duplicate Rows

1. `df = df.drop_duplicates()`

- Keeps only the **first occurrence** and removes all duplicates.

Correlations (Finding Relationships Between Data)

What is Correlation?

Correlation tells us **how two columns are related**:

- ✓ **Positive correlation** → If one value increases, the other also increases.
- ✓ **Negative correlation** → If one value increases, the other decreases.
- ✓ **No correlation** → No clear relationship between values.

Example: Check Correlation in a DataFrame

```
1. print(df.corr())
```

- This gives a **correlation table**, showing how each column is related to others.

Example Output:

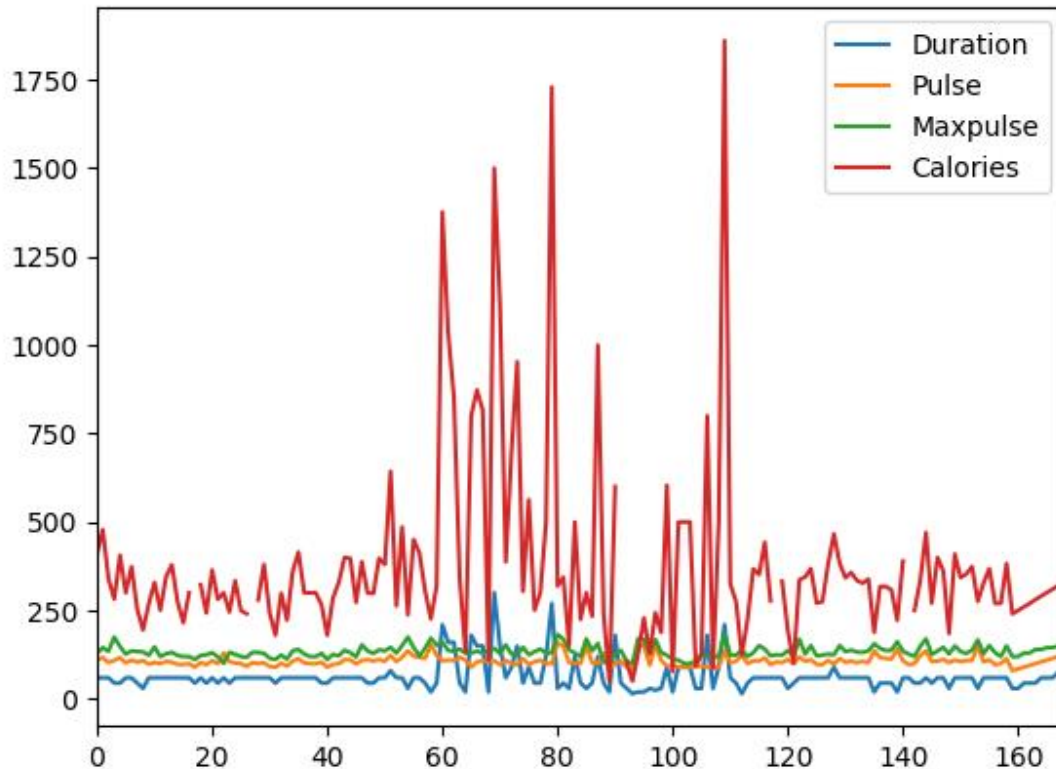
	Duration	Pulse	Maxpulse	Calories
Duration	1.000000	0.520	0.412	0.893
Pulse	0.520	1.000	0.760	0.635
Maxpulse	0.412	0.760	1.000	0.490
Calories	0.893	0.635	0.490	1.000

- ✓ 0.893 between **Duration & Calories** → Strong positive correlation (longer workouts burn more calories).
- ✓ 0.412 between **Maxpulse & Duration** → Weak correlation.

Summary: What We Learned

- ✓ **Remove empty cells** → `dropna()`
- ✓ **Fill empty cells** → `fillna(value)`
- ✓ **Fix wrong formats** → `to_datetime()` and `to_numeric()`
- ✓ **Remove duplicates** → `drop_duplicates()`
- ✓ **Find correlations** → `df.corr()`

Pandas - Plotting



Basic Plotting in Pandas

Pandas uses the **plot()** method to create diagrams.

Example: Simple Line Plot

1. `import pandas as pd`
 2. `import matplotlib.pyplot as plt`
 3. `df = pd.read_csv('data.csv')` # Load the dataset
 4. `df.plot()` # Create a default line plot
 5. `plt.show()` # Display the plot
- **plot()** creates a basic **line graph** using all numerical columns.

Scatter Plot (Showing Relationships)

A **scatter plot** is useful for showing the **relationship between two columns**.

We must **specify** which column will be on the x-axis and which will be on the y-axis.

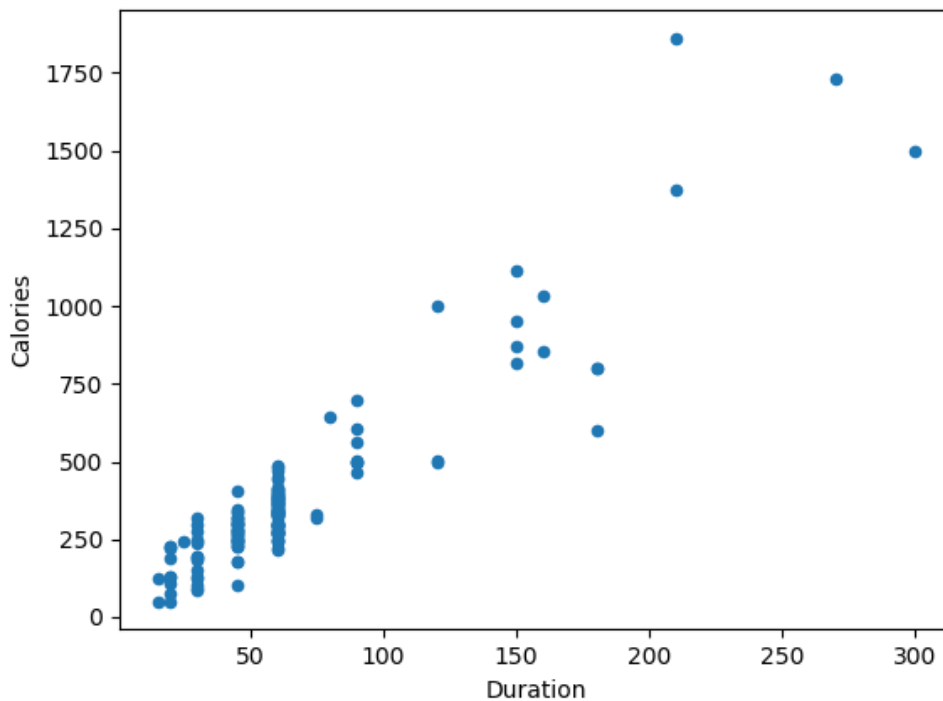
Example: Scatter Plot for "Duration" vs "Calories"

1. `df.plot(kind='scatter', x='Duration', y='Calories') # Scatter plot`
2. `plt.show()`

- If the points form a **pattern**, it means there is a **relationship** between the two columns.

✓ Higher "Duration" → More "Calories" burned (positive correlation).

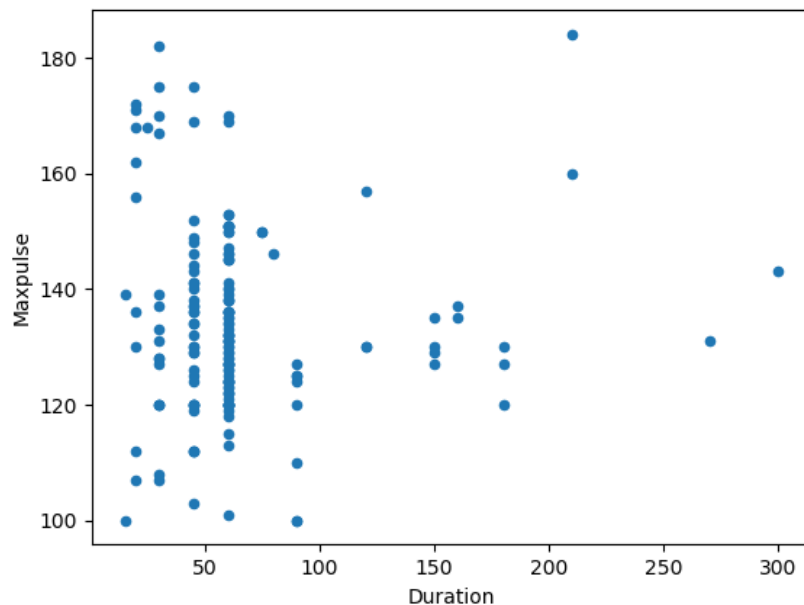
✓ This confirms our earlier analysis!



Example: Scatter Plot for "Duration" vs "Maxpulse" (No Relationship)

1. `df.plot(kind='scatter', x='Duration', y='Maxpulse')`
2. `plt.show()`

- If the points are **randomly spread**, it means there is **no strong relationship** between the columns.



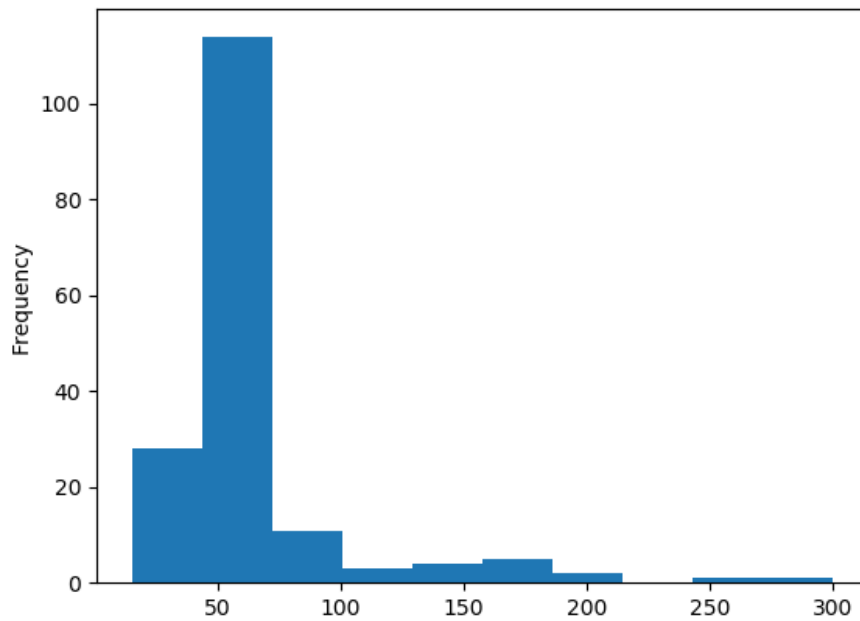
Histogram (Showing Frequency of Values)

- A **histogram** helps us see how data is **distributed**.
- For example, we can check how many workouts lasted **between 50 and 60 minutes**.

Example: Histogram for "Duration"

1. `df["Duration"].plot(kind='hist')` # Histogram for workout duration
2. `plt.show()`

- **The taller bars mean more data falls in that range.**
✓ If most workouts are between **40-60 minutes**, the **bar** for that range will be the **tallest**.



Summary: What We Learned ✓

✓ **Line Plot** → `df.plot()` → Shows trends over time.

✓ **Scatter Plot** → `df.plot(kind='scatter', x='col1', y='col2')` → Shows relationships between two columns.

✓ **Histogram** → `df["col"].plot(kind='hist')` → Shows how often values occur in a dataset.